

CALIBRATION-BASED DEVELOPMENT OF A WEB-OPERATED DYNAMIXEL HEXAPOD ROBOT

Wong Zhen Hao^{1}, Tiong Yew Tang¹, Chian C. Ho², Kian Meng Yap¹*

¹*Research Centre For Human-Machine Collaboration (HUMAC), Department of Smart Computing and Cyber Resilience, Faculty of Engineering and Technology, Sunway University, Bandar Sunway, Selangor, Malaysia*

²*Department of Electrical Engineering, National Yunlin University of Science and Technology, Yunlin County, Taiwan*

**wzhenhao06@gmail.com*

Keywords: HEXAPOD ROBOT, DYNAMIXEL ACTUATOR, WEB-BASED CONTROL, RASPBERRY PI, READY-POSE CALIBRATION

Abstract

This paper presents the calibration-based development of a web-operated Dynamixel hexapod robot as a locomotion-control subsystem within a broader Autonomous Palm Oil Harvesting Robot project. The original robot relied on a CM-530 and predefined RoboPlus Motion pages for movement and control, which limited the flexible integration with live actuator control, Raspberry Pi deployment as an onboard computer, controller input, inverse kinematics, and live actuator feedback and diagnostics. The proposed system converts the platform into a Python-controlled architecture using an onboard Raspberry Pi server, web interface, controller-client communication, and direct Dynamixel command routing. A key contribution is the repeatable calibration method that maps actuator IDs to physical joints, separates code-perfect reference values from the measured physical-flat pose, and records ready-pose offsets caused by horn alignment, actuator orientation, and a variety of mechanical factors. The system was tested on an 18-actuator hexapod using AX-series Dynamixel motors and supports ready pose, walking, strafing, turning, body-height adjustment, push-up motion, motor health checking, fixed gait fallback, and inverse-kinematics-based movement testing. Results show that stable hexapod control requires both software command structure and physical calibration, especially under real actuator load, voltage, temperature, and mechanical alignment constraints.

1 Introduction

Hexapod robots are useful and reliable platforms for studying legged locomotion, gait coordination, and hardware-aware robotic control [1]. Compared to wheeled platforms, a hexapod can maintain body support while selected legs are lifted which can support gait-based movement, but practical control is more complex because multiple joints must move in a coordinated and repeatable manner.

The robot used in this work is based on ROBOTIS AX-series Dynamixel smart actuators [2], and was previously operated through a CM-530 and RoboPlus Motion workflow for pre-programmed motion pages and control. Although this motion-page approach is suitable for fixed demonstrations, it limits the flexibility of integration with live actuator control and feedback, Raspberry Pi deployment, web operation, controller input, inverse kinematics, and actuator diagnostics.

This paper presents a calibrated Python-based control bridge that maps actuator IDs to physical joints, stores ready-pose values, routes web and controller commands, monitors actuator feedback, and supports fixed gait fallback with inverse-kinematics-based movement testing [3]. The aim is not to claim a fully optimised autonomous locomotion algorithm, but to document a practical hardware-control foundation that can support the broader Autonomous Palm Oil Harvesting Robot project through future gait refinement, ROS-compatible integration, reinforcement learning, and safer simulation-to-real transfer.

2 Methodology

The proposed system was developed using a hardware-in-the-loop approach, where each major function was tested directly on the physical hexapod instead of relying only on simulation or predefined motion pages. This method of development allowed the control system to be evaluated under real operating conditions, including actuator temperature, floor contact, friction, and other hardware conditions that can affect the performance of the hexapod. The platform uses 18 ROBOTIS AX-series Dynamixel actuators arranged across six legs, with each leg consisting of three joints: coxa, femur, and tibia. The original CM-530 and Dynamixel bus structure were retained, while Python control was added through a Raspberry Pi server and Dynamixel communication interface.

The control bridge separates high-level commands from low-level actuator execution. Commands from the web interface, terminal, or controller-client script are received by the Raspberry Pi server and routed through IKControl.py. The controller manages motor mapping, ready-pose execution, fixed gait fallback, inverse-kinematics-based movement testing, actuator feedback reading, and diagnostic checks before sending Dynamixel raw-position commands to the actuator bus.

The overall software command flow is shown in Fig. 1.

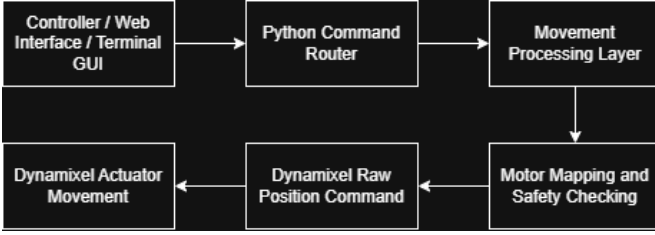


Fig. 1 Software command flow from web and controller input to Dynamixel actuator execution.

A repeatable calibration method was developed and used to improve movement consistency. With this method, each Dynamixel ID was mapped to its physical joint, a code-perfect reference pose was defined, and the robot was aligned in a flat star-shaped posture for horn setup. The measured physical-flat motor positions were then recorded and used to update the ready-pose dictionary. During movement testing, actuator position, voltage, temperature, load, and movement state were monitored to identify unstable joints, under and overcompensating legs, excessive load, overheating, and communication problems.

3 Results and Discussion

The developed system changed the hexapod from a CM-530/RoboPlus motion-page-based demonstration platform into a programmable Python-controlled experimental platform. IKControl.py became the main controller and supports web control, terminal control, controller-client communication, fixed gait fallback, IK Motion, IK Bézier Motion, ready-pose execution, and actuator health checking. The calibration process also improved the repeatability of the robot's starting posture by separating the code-perfect reference pose, measured physical-flat pose, and final calibrated ready pose.

To provide preliminary quantitative validation, a five-cycle walking test was performed using the same speed setting, starting line, floor surface, and front-body measurement reference. The robot was returned to the calibrated ready pose before each test, and actuator temperature was recorded before and after each five-cycle run. The timing uncertainty was approximately ± 0.8 s due to manual command and stopping latency.

Table 1 Preliminary five-cycle movement comparison at speed setting 50

Mode	Time (s)	Distance (cm)	Speed (cm/s)	Max Temp. (°C)
Fixed gait	46.06	57.0	1.24	33→43
IK Motion	23.80	63.0	2.65	40→43
IK Bézier	23.40	62.5	2.67	36→44

The fixed gait fallback produced the steadiest movement, but required more time to complete the same five gait cycles because of its more conservative gait timing. IK Motion and IK Bézier Motion completed the test faster and produced slightly longer forward displacement, showing more efficient

movement under the tested speed setting. However, these results are treated as preliminary physical validation rather than a complete gait-efficiency benchmark, because battery condition, floor contact, actuator load, and starting temperature can affect the measured values. The tests also confirmed that actuator temperature and load must be monitored during repeated movement, and that the developed bridge provides a more flexible and observable control layer than the original motion-page baseline.

4 Conclusion

This paper presented the development of a calibrated Python-based control bridge for a web-operated Dynamixel hexapod robot. The system transformed the original CM-530 and RoboPlus motion-page platform into a more programmable experimental robot by adding joint-level command routing, onboard Raspberry Pi operation, web control, controller-client communication, inverse-kinematics-based movement testing, and live actuator diagnostic feedback.

The main finding is that stable hexapod control depends not only on movement code, but also on repeatable hardware calibration and actuator observation. Moreover, the separation between the code-perfect reference pose, measured physical-flat pose, and final calibrated ready pose helped document offsets caused by horn placement, actuator orientation, bracket tolerance, and mechanical loading. As a result, the completed system provides a practical foundation for improved gait generation, ROS-compatible integration, computer vision, reinforcement learning, and safer simulation-to-real transfer.

5 Acknowledgements

The authors wish to acknowledge HUMAC Lab and the Faculty of Engineering and Technology, Sunway University, for their support, facilities, and research environment throughout the development and testing of the hexapod robot. We also wish to acknowledge the support of the Sunway Advanced Computing Facility (SACF) for cloud computing resources that enabled the success of this research.

6 References

- [1] Ma, J., Qiu, S., Yang, J., et al.: 'Design, analysis and experiments of hexapod robot with six-link legs for high dynamic locomotion', *Micromachines*, 2022, 13, (9), 1404
- [2] ROBOTIS.: 'DYNAMIXEL SDK', ROBOTIS e-Manual, available at: https://emanual.robotis.com/docs/en/software/dynamixel/dynamixel_sdk/, accessed 2 July 2026
- [3] Lizarraga, J.A., Garnica, J.A., Ruiz-Leon, J., et al.: 'Advances in the kinematics of hexapod robots: An innovative approach to inverse kinematics and omnidirectional movement', *Applied Sciences*, 2024, 14, (18), 8171